

머신러닝 입문

— 데이터로 배우고, 왜 해석 가능해야 하는가

기계가 데이터에서 스스로 규칙을 배우는 법, 그리고 GPT 시대에도 **해석 가능한** 머신러닝을 배우는 이유

머신러닝은 규칙을 사람이 짜는 대신 **데이터에서 규칙을 배우게** 하는 기술입니다. 이 강의는 (1) 머신러닝이 무엇이고 AI·딥러닝과 어떻게 다른지, (2) 지도·비지도·강화학습의 세 갈래, (3) 대출·진단 같은 고위험 결정에서 왜 **해석 가능한** 전통 머신러닝이 필요한지를 다룹니다. 이 과정에서 배울 회귀·트리·SVM·KNN이 모두 그 해석 가능한 모델들이에요.

이 강의를 마치면 할 수 있는 것

네 가지를 목표로 합니다.

01

머신러닝이 무엇인지 말한다

규칙을 **직접 짜는** 전통 프로그래밍과, 데이터에서 **규칙을 배우는** 머신러닝의 차이를 설명할 수 있다.

02

AI·ML·DL을 구분한다

인공지능 > 머신러닝 > 딥러닝의 포함 관계를 알고, LLM이 그중 어디에 있는지 안다.

03

세 가지 학습을 안다

지도·비지도·강화학습을 구분하고, 이 과정이 다루는 지도학습(분류·회귀)이 무엇인지 안다.

04

왜 해석 가능한 ML인가에 답한다

고위험 결정에 **왜 해석 가능성**이 필요한지(신뢰·규제·공정성)와, 그걸 얻는 두 길(내재적·사후 — SHAP)을 설명할 수 있다.

기계가 '배운다'는 건 무슨 뜻일까

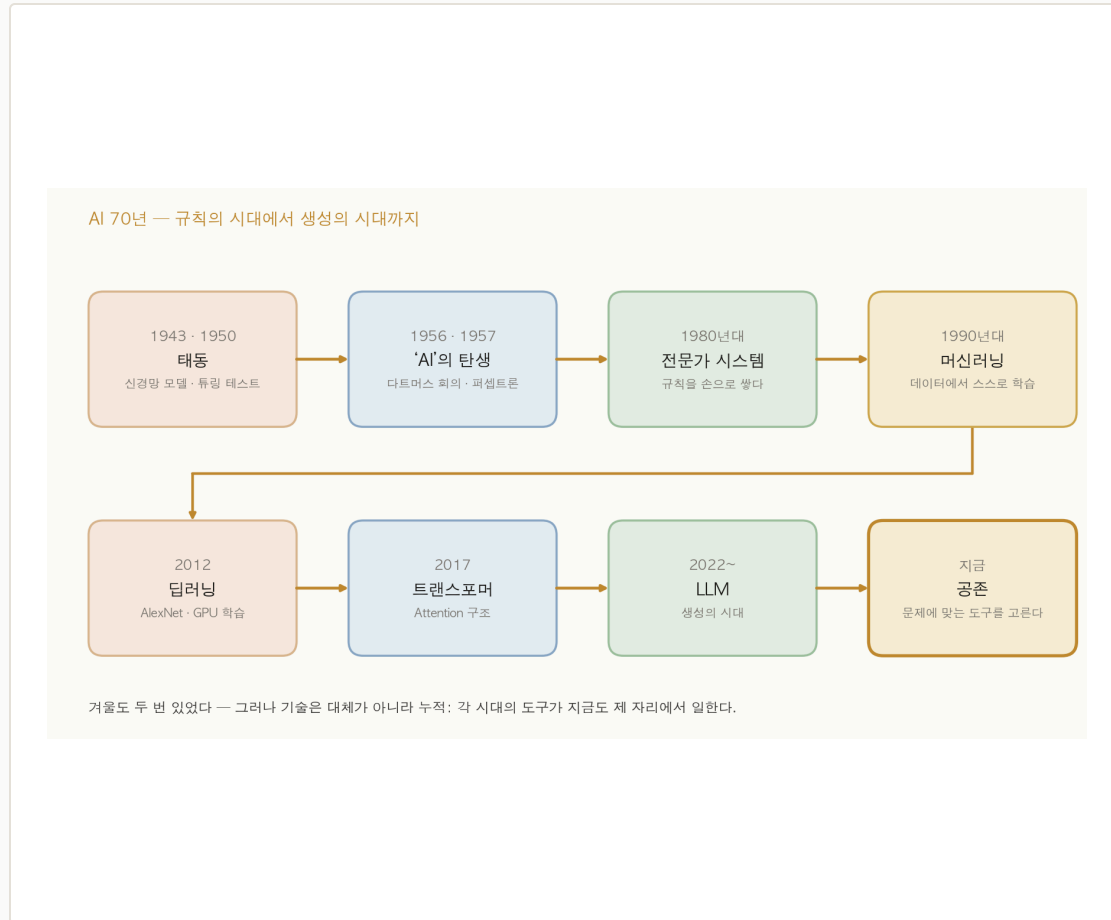
GPT가 글도 쓰고 그림도 그리는 시대 — 그런데 '머신러닝'은 정확히 무엇이고, 왜 지금도 배워야 할까요? 출발점은 단순한 질문입니다: **기계가 '배운다'는 게 무슨 뜻일까?**

규칙을 짜는 대신, 규칙을 배우게 한다. 전통적인 프로그래밍은 사람이 규칙을 일일이 짭니다 — "제목에 '광고'가 있으면 스팸"처럼요. 하지만 예외가 끝없이 생기죠. 머신러닝은 발상을 뒤집습니다: **스팸이라고 표시된 메일 수만 통을 주고, 규칙을 스스로 찾게** 합니다.

그래서 데이터가 핵심입니다. 머신러닝 모델은 데이터 속 **패턴**을 학습해 새 데이터에도 적용합니다. 사람이 규칙을 못 쓰던 문제 — 이미지 인식, 음성 인식, 추천 — 가 이렇게 풀렸어요.

이 강의의 지도. 먼저 머신러닝이 **무엇인지**(AI·딥러닝과의 관계, 학습의 세 갈래)를 보고, 그다음 **왜 해석 가능한 머신러닝**이 고위험 결정에 필요한지를 봅니다 — 이 과정에서 배울 회귀·트리·SVM이 바로 그 모델들이에요.

AI 70년 — 규칙의 시대에서 데이터의 시대로



01 태동과 'AI'의 탄생 (1943~1957)

맥컬러·피츠의 신경망 이론(1943), 튜링 테스트(1950), 다트머스 회의에서 'AI'라는 말이 처음 쓰였고(1956), 퍼셉트론(1957)이 등장했습니다.

02 규칙의 시대 (1980년대)

전문가의 지식을 **규칙으로 직접 코딩**하는 전문가 시스템이 유행했습니다 — 하지만 규칙을 일일이 짜는 데는 한계가 있었죠.

03 데이터의 시대 — 머신러닝 (1990s~)

규칙을 짜는 대신 **데이터에서 학습**하는 머신러닝이 부상했고, 2012년 딥러닝 (AlexNet)·2017년 트랜스포머로 폭발했습니다.

04 생성의 시대 (2022~)

트랜스포머에서 LLM이 나와 '생성의 시대'가 열렸어요. 그래도 전통 ML은 사라지지 않았습니다 — 다음 장들에서 왜인지 봅니다.

1943년 신경망 이론부터 2020년대 LLM까지. 기술은 대체가 아니라 누적 — 각 시대의 도구가 지금도 제 자리에서 일한다.

규칙을 짜는 일에서, 규칙을 '배우게' 하는 일로



전통 프로그래밍은 규칙을 입력해 답을 얻고, 머신러닝은 데이터와 답을 주면 규칙(모델)을 산출한다 — 입력과 출력이 뒤집힌다.

01 전통 프로그래밍 — 규칙이 입력

사람이 규칙을 짜 넣으면("if '광고' in 제목: 스팸"), 새 데이터에 그 규칙을 적용해 답을 냅니다. 예외가 생길 때마다 규칙을 더 써야 해요.

02 머신러닝 — 규칙이 출력

데이터 + 답(라벨)을 주면, 학습 알고리즘이 패턴을 찾아 규칙(=모델)을 만들어냅니다. 새 데이터도 그 모델로 처리하죠.

03 그래서 '학습'이다

사람이 규칙을 못 쓰는 문제(사진 속 고양이, 손글씨)도, 예시만 충분하면 모델이 규칙을 스스로 찾습니다.

04 대가 — 설명이 어려워진다

규칙을 사람이 안 짜니, 왜 그렇게 판단했는지가 모델 안에 숨습니다. 복잡한 모델일수록 더 그렇죠 — 뒤에서 다룰 '해석 가능성' 문제예요.

AI ⊃ 머신러닝 ⊃ 딥러닝 — 그리고 LLM의 자리



인공지능이 가장 넓고, 머신러닝은 그 한 방법, 딥러닝은 머신러닝의 한 도구다. LLM은 딥러닝의 한 종류 — 'ML 대 LLM'은 잘못된 대립이다.

01 AI — 가장 넓은 목표

인공지능은 '사람처럼 지능적인 행동을 하는 시스템' 전부를 가리키는 가장 넓은 말입니다.

02 ML — 데이터로 배우는 방법

머신러닝은 그 AI를 구현하는 한 방법 — 규칙을 코딩하지 않고 **데이터에서 패턴을 학습**합니다.

03 DL — 깊은 신경망 도구

딥러닝은 머신러닝의 한 갈래로, **깊은 신경망**으로 표현까지 스스로 학습합니다. GPT 같은 LLM이 여기 속해요.

04 그래서 LLM도 ML의 일부

LLM은 머신러닝 밖의 무언가가 아니라 **그 안의 한 종류**입니다. 올바른 질문은 'ML이냐 LLM이냐'가 아니라 '**어떤 문제에 어떤 도구냐**'예요.

기계는 어떻게 배우나 — 학습의 세 갈래

머신러닝은 무엇을 주고 무엇을 배우게 하느냐에 따라 크게 셋으로 나뉩니다.

지도

정답을 주고 배운다

지도학습 — 라벨(정답)이 달린 데이터로 학습합니다. "이건 스팸, 저건 정상"을 주면 새 메일을 분류하죠. **분류**와 **회귀**가 여기 속하고, 이 과정의 무대입니다.

비지도

정답 없이 구조를 찾는다

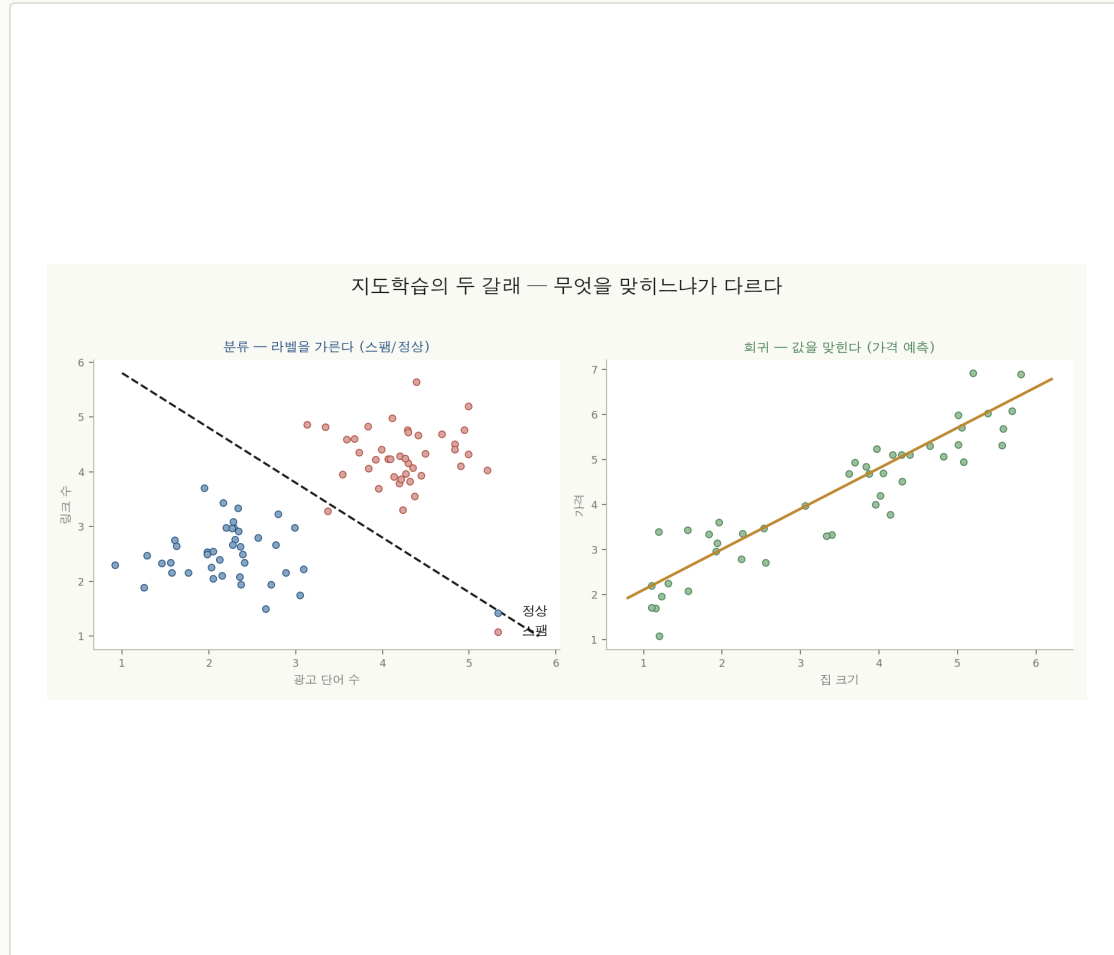
비지도학습 — 라벨 없는 데이터에서 **패턴·그룹**을 스스로 찾습니다. 비슷한 고객을 묶는 **군집화**, 변수를 줄이는 **차원 축소**가 대표적이에요.

강화

시행착오로 보상을 쫓는다

강화학습 — 행동하고 **보상**을 받으며 시행착오로 최선의 전략을 배웁니다. 게임·로봇 제어처럼 '결정의 연속'인 문제에 쓰여요.

이 과정의 무대 — 지도학습: 분류와 회귀



라벨이 있는 데이터로 학습한다. 분류는 **어느 쪽인지**(스팸/정상)를, 회귀는 **얼마인지**(가격)를 맞힌다.

01 라벨 = 정답

지도학습은 입력마다 **정답(라벨)**이 붙은 데이터로 배웁니다 — 스팸 여부, 실제 집값처럼요.

02 분류 — 어느 쪽인가

정해진 **보기 중 하나**를 고릅니다(스팸/정상, 개/고양이). 로지스틱 회귀·트리·SVM·KNN이 분류 모델이에요.

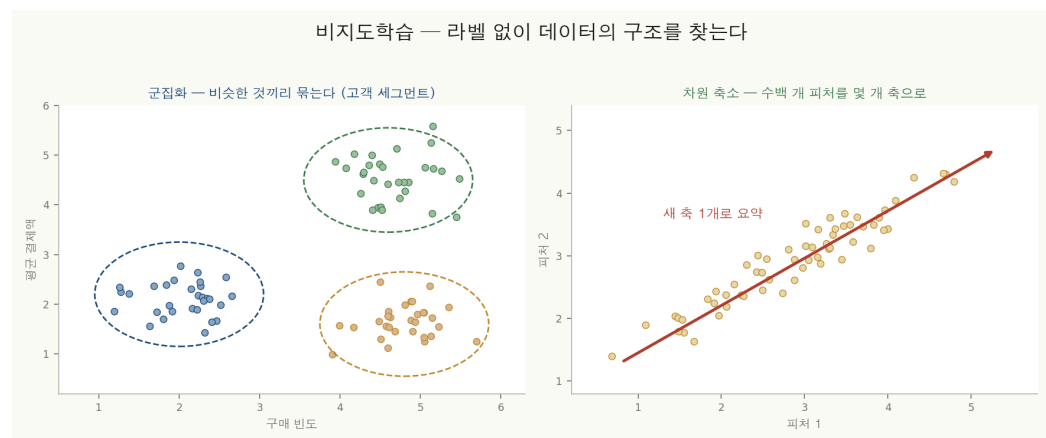
03 회귀 — 얼마인가

연속된 숫자를 맞힙니다(가격, 기온). 선형 회귀가 대표적 — 바로 다음 모듈입니다.

04 이 과정이 다루는 것

회귀 → 로지스틱 → 트리·랜덤포레스트 → SVM → KNN. 모두 지도학습이고, 모두 **해석 가능한** 모델이에요.

정답 없이 구조를 찾는다 — 비지도학습



라벨 없는 데이터에서 패턴을 찾는다. 군집화는 비슷한 것끼리 묶고, 차원 축소는 많은 변수를 핵심 축으로 요약한다.

01 군집화 (Clustering)

비슷한 데이터끼리 **그룹으로 묶습니다**. 고객을 행동 패턴별로 나누는 **세그먼트**가 대표적이에요 — 정답을 안 주고 구조만 찾죠.

02 차원 축소 (Dimensionality Reduction)

수백 개 변수를 **몇 개의 핵심 축**으로 요약합니다(PCA 등). 시각화·노이즈 제거·전처리에 쓰여요.

03 왜 라벨이 없나

현실 데이터 대부분은 라벨이 없습니다 — 라벨링은 비싸니까요. 비지도학습은 **날것의 데이터**에서 바로 통찰을 얻습니다.

04 이 과정의 범위

이 입문 과정은 **지도학습**에 집중하지만, 비지도학습이 또 다른 큰 갈래임을 알아두면 좋아요.

그냥 LLM에게 맡기면 되지 않나?

여기까지 보면 의문이 듭니다 — GPT 같은 LLM이 이렇게 강력한데, 굳이 선형 회귀나 결정 트리 같은 전통 ML을 배워야 할까요?
"고위험 결정도 LLM에게 물어보면 되지 않나?"

머신러닝 모델은 대출 승인, 질병 진단, 입사 서류 선별을 **대신 결정**합니다. 그런데 이런 고위험 결정에는 결과만큼이나 **"왜 그렇게 판단했는가"**가 중요해요. 대출이 거절된 고객은 사유를 알 권리가 있고, 의사는 진단 보조 모델이 **무엇을 보고** 그 결론에 닿았는지 확인해야 합니다.

해석 가능성(interpretability)은 모델의 예측을 사람이 이해할 수 있는 정도입니다. 그리고 이 "왜"를 답하는 능력이야말로 거대한 블랙박스인 **LLM**이 **가장 못하는 것**이에요.

그래서 "그냥 LLM 쓰면 되지"라는 생각은 바로 여기서 막힙니다. **강력함과 설명 가능성**은 다른 축이거든요 — 다음 장에서 둘을 정면으로 비교합니다.

고위험 결정을 누구에게 맡길까 — LLM vs 해석 가능한 모델

대출 거절을 맡긴다고 해봅시다. 강력한 LLM이 있는데도, '왜'를 설명해야 하는 결정엔 LLM을 그대로 쓸 수 없습니다 — 바로 해석 가능성 때문이에요.

그냥 LLM에게 맡기면

LLM (생성형 AI)

이유가 진짜인지 보장이 없다

LLM이 그럴듯한 사유를 댕니다. 하지만 그 '사고 과정'이 **실제 내부 계산과 어긋난다**는 연구가 있어요 — 사후에 지어낸 합리화일 수 있습니다.

재현·감사가 안 된다

수천억 파라미터의 **블랙박스**입니다. 같은 신청서에 다른 답이 나오기도 하고, 결정 근거를 **증명**할 수 없어요.

규제를 못 넘는다

EU AI Act·GDPR이 요구하는 '설명'을 제공하지 못합니다 — 고위험 분야에선 **채택 불가**.

해석 가능한 모델에게 맡기면

전통적 ML (선형·트리)

검증 가능한 이유를 댕다

"부채/소득 비율 계수가 기준을 넘어 거절"처럼 **계수·규칙**으로 근거를 제시합니다 — 사람이 따라 읽고 검증할 수 있어요.

재현·감사가 된다

같은 입력엔 **항상 같은 답**. 피처별 기여를 열어 검증·디버깅할 수 있습니다.

규제를 충족한다

설명 의무·거절 사유 고지를 그대로 만족합니다. **바로 쓸 수** 있어요.

해석 가능성이 필요한 세 가지 이유

그 '설명해야 하는 이유'는 결국 세 가지로 정리됩니다 — 해석 가능성이 **없으면 곤란한** 상황들이에요.

신뢰

사람이 결과를 받아들이려면

이해관계자가 모델을 신뢰하려면 근거가 필요합니다. "모델이 그랬다"로는 의사·고객·심사자를 설득할 수 없어요. 예측이 빗나갔을 때 **원인을 찾아 고치는** 디버깅도 근거가 있어야 가능합니다.

규제

법이 설명을 요구한다

EU AI Act(고위험 2026.8 전면 적용)와 GDPR은 자동화된 결정에 대한 **설명**을 요구합니다. 설명할 수 없으면 그 분야에선 쓸 수 없어요.

공정성

편향을 잡아내려면

모델이 성별·인종 같은 민감 정보를 **대리 변수**로 학습했는지 확인하려면 예측의 근거를 들여다봐야 합니다. 편향은 근거를 열어 봐야 드러나요.

설명할 수 없는 모델이 위험해지는 곳

해석 가능성이 추상적 이상이 아니라 **현실의 요건**이 되는 분야가 있습니다 — 사람의 삶에 직접 영향을 주는 고위험 결정입니다.

편향 사례 — COMPAS. 미국 형사사법에서 재범 위험을 예측하던 모델이 인종에 따라 편향된 점수를 냈다는 보도가 있었습니다(ProPublica, 2016). 블랙박스였기에 **무엇이 잘못됐는지** 따지기 어려웠어요.

규제 — EU AI Act. 신용·채용·의료 같은 고위험 AI에 투명성을 의무화했고, 고위험 관련 규정은 **2026년 8월부터 전면 적용**됩니다(EU Regulation 2024/1689). GDPR도 자동화된 결정에 대한 설명을 다룹니다(제22조).

신용 — 거절 사유 고지. 대출이 거절되면 그 **구체적 사유**를 알려야 하는 제도가 여러 나라에 있습니다. 예측 근거를 못 대는 모델은 이 요건을 충족하지 못해요.

그래서 고위험 분야에서는 성능이 조금 높아도 **설명할 수 없으면 채택되지 못합니다.** 해석 가능성이 곧 **사용 가능성**인 셈이에요.

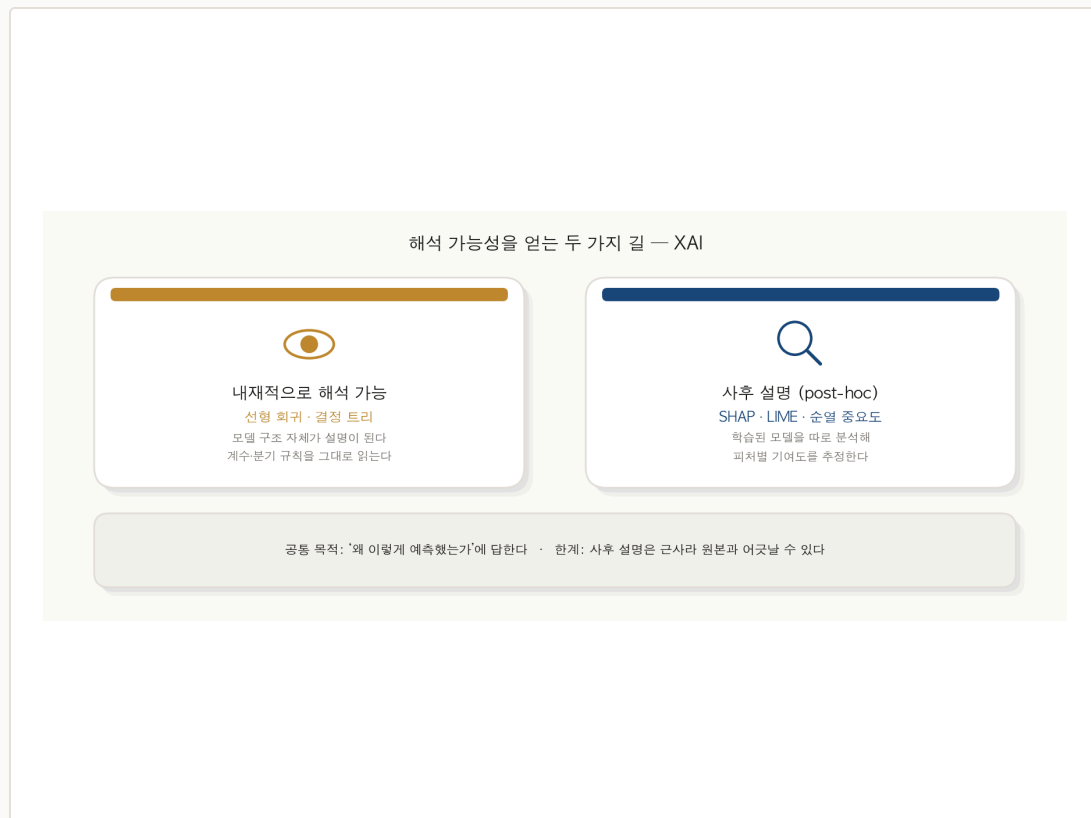


EU AI Act — 고위험 AI의 투명성·설명 의무. 고위험 규정은 2026년 8월부터 전면 적용. (EU Regulation 2024/1689)

고위험 분야에서 해석 가능성은
선택이 아니라 사용의 전제 조건이다.

해석 가능성을 얻는 두 가지 길 — 내재적과 사후

해석 가능성을 얻는 방법은 크게 둘입니다. 모델 자체를 **처음부터 해석 가능하게** 만들거나(내재적·glass-box), 복잡한 블랙박스를 **학습한 뒤 별도 기법으로 설명**하거나(사후·post-hoc).



01 내재적(glass-box)

선형·로지스틱 회귀, 결정 트리처럼 **구조 자체가 설명**이 되는 모델입니다. 추가 도구 없이 계수·규칙을 그대로 읽어요 — 이 과정에서 배울 모델들이죠.

02 사후(post-hoc)

이미 학습된 복잡한 모델을 **밖에서 분석**해 설명을 만듭니다. SHAP·LIME이 대표적이에요(다음 장).

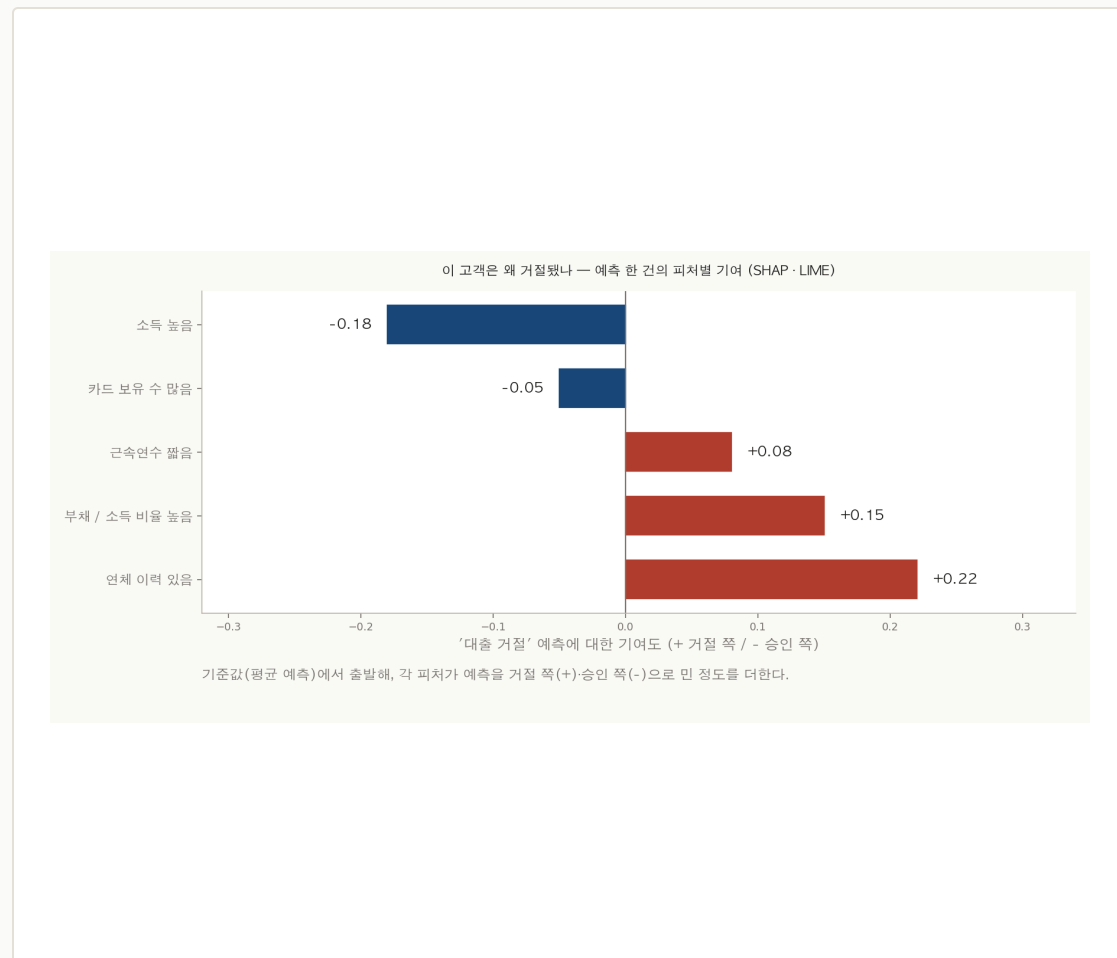
03 전역 vs 지역

모델 **전체**의 경향을 보는 설명(순열 중요도 등)과, 예측 **한 건**의 이유를 보는 설명(SHAP·LIME)으로도 나뉩니다.

04 고위험엔 내재적부터

사후 설명은 **근사**라 원본과 어긋날 수 있어, 고위험 결정엔 처음부터 해석 가능한 모델을 쓰자는 입장이 힘을 얻습니다(Rudin, 2019).

SHAP과 LIME — 예측 한 건의 근거를 숫자로



한 예측의 피처별 기여를 더해 결과를 설명한다 — 양수는 거절 쪽, 음수는 승인 쪽으로 민 정도.

01 LIME — 국소 근사

예측 한 점 주변에서 복잡한 모델을 단순한 선형 모델로 근사해 설명합니다(Ribeiro 외, 2016). 어떤 모델에도 붙일 수 있어요.

02 SHAP — 게임이론 기반

Shapley 값(여럿이 함께 낸 성과를 각자 몫으로 공정히 나누는 게임이론 방법)으로 각 피처의 기여를 분배합니다(Lundberg & Lee, 2017). 일관성이 보장돼 실무 표준이에요.

03 표 데이터의 단골

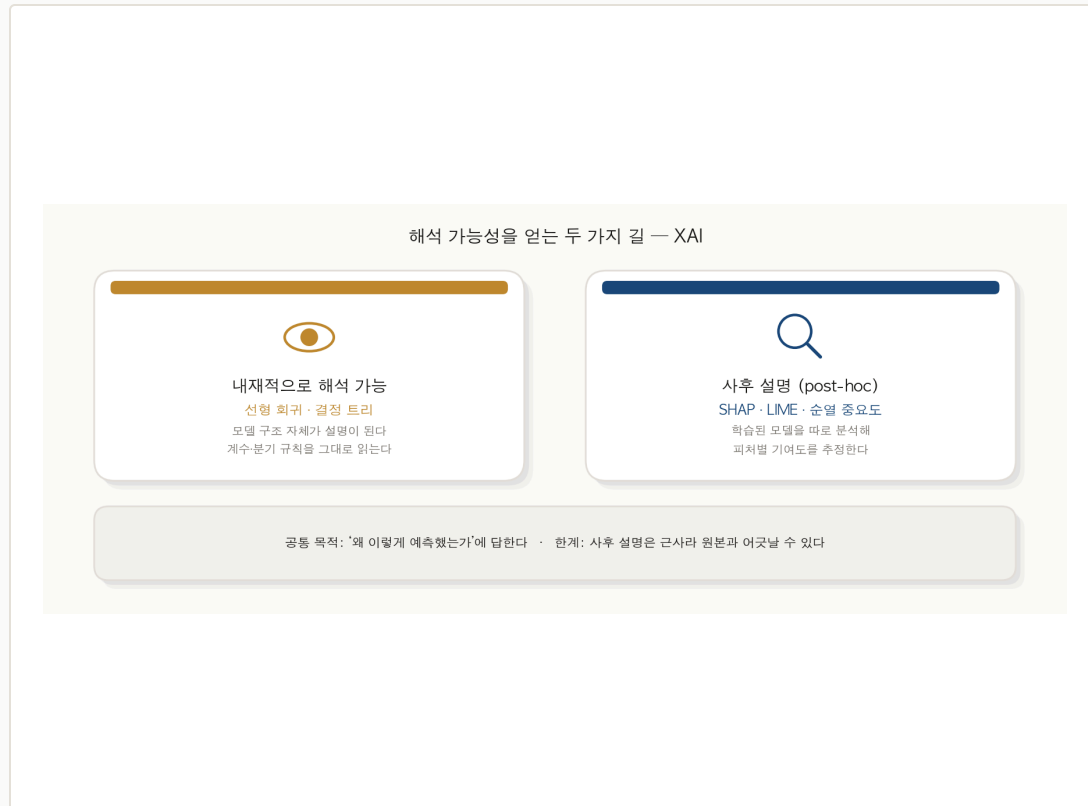
트리 계열엔 정확하고 빠른 TreeSHAP이 있어, 부스팅 트리 + SHAP 조합이 표 데이터 실무에서 널리 쓰입니다.

04 출력은 '기여도'

기준값에서 출발해 각 피처가 예측을 어느 쪽으로 얼마나 밀었는지를 더해 보여줍니다 — "이 고객은 왜 거절됐나"에 답하죠.

그래서 전통 머신러닝부터 — 해석이 기본인 모델들

해석 가능성을 **직접 익히기**엔 전통적 머신러닝만 한 게 없습니다. 선형 모델의 계수, 트리의 규칙은 **그 자체로 설명**이라 별도 기법 없이도 "왜"를 읽을 수 있어요. 이 과정이 다루는 모델들이 바로 그것입니다.



01 회귀·로지스틱 회귀 — 계수

각 피쳐의 계수로 "무엇이 결과를 어느 쪽으로 미는지"를 바로 읽습니다. 다음 모듈이 회귀예요.

02 결정 트리·랜덤 포레스트 — 규칙·중요도

예측을 규칙의 연쇄로 따라 읽고, **변수 중요도**로 영향력을 봅니다.

03 SVM·KNN — 경계와 이웃

경계를 긋거나 가까운 이웃에 묻는 방식으로, 동작을 사람이 따라갈 수 있어요.

04 그리고 SHAP으로 확장

이 모델들에 SHAP을 붙이면 **예측 한 건**의 근거까지 봅니다 — 내재적·사후 해석을 한 번에 익히는 셈이에요.

정리 — 머신러닝, 그리고 왜 해석 가능한 모델인가

오늘 다룬 것

- ▶ **머신러닝** — 규칙을 짜는 대신 데이터에서 규칙(모델)을 배운다
- ▶ **AI ⊃ ML ⊃ DL** — LLM도 딥러닝의 한 종류, ML의 일부
- ▶ **세 갈래** — 지도(분류·회귀)·비지도(군집·추소)·강화학습
- ▶ **왜 해석 가능** — 신뢰·규제(EU AI Act 2026.8)·공정성
- ▶ **두 길** — 내재적(선형·트리) vs 사후(SHAP·LIME)

이 과정에서 배울 것

- ▶ **회귀** — 숫자를 예측하는 첫 모델 (다음 모듈)
- ▶ **로지스틱 회귀** — 직선을 확률로 바꾸기
- ▶ **결정 트리·랜덤 포레스트** — 규칙과 변수 중요도
- ▶ **SVM·KNN** — 경계와 이웃으로 분류
- ▶ **공통점** — 모두 지도학습, 모두 해석 가능한 모델

머신러닝은 데이터에서 규칙을 배우는 기술이고, 고위험 결정에서는 '왜 그렇게 판단했는가'를 설명할 수 있어야 한다. 그래서 이 과정은 해석 가능한 전통 머신러닝 — 회귀·트리·SVM·KNN — 을 배운다. 다음은 그 출발점, 회귀다.